



7



12



1800 sp

عملي

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 2025/12/2

كلية الهندسة المعلوماتية

السنة الرابعة

ARRAYS

عملي مشترك



نظم التشغيل

المصفوفات (Arrays)

- المصفوفة هي هيكل بيانات يُستخدم لتخزين قيم متعددة تحت اسم متغير واحد.
- يمكن الوصول إلى كل قيمة باستخدام الفهرس الرقمي (index) أو المفتاح النصي (key).
- تساعد المصفوفات في إدارة قوائم البيانات بسهولة مقارنة باستخدام متغيرات منفصلة لكل قيمة.

يمكن أن نستخدم المصفوفات من أجل:

1. تجنب إنشاء العديد من المتغيرات المنفصلة مثل (name1, name2, name3...).
2. إمكانية المرور على القيم بسهولة باستخدام الحلقات (loops).
3. تخزين البيانات المهيكلة مثل معلومات المستخدم أو إعدادات config.
4. جعل الـ scripts أكثر وضوحاً وسهولة في الصيانة (maintainable).

أنواع المصفوفات في Bash

1. المصفوفة المفهرسة (Indexed Array):

يتم ترقيم العناصر تلقائياً بدءاً من الصفر.

```
arr=("apple" "banana" "cherry")
echo ${arr[1]} # banana
```

2. المصفوفة الترابطية أو المرتبطة بالمفتاح (Associative Array - Keyed)

يتم تخزين العناصر باستخدام مفاتيح نصية، تشبه القواميس (Dictionary).

طريقة تعريفها: declare -A array_name

طريقة إسناد القيم: array_name[key]=value

(يجب الانتباه إلى عدم وجود فراغات قبل أو بعد علامة المساواة (=) عند الإسناد، وإلا سيؤدي ذلك إلى خطأ في التنفيذ)

```
declare -A user
user[name]="Mohammad"
user[role]="Cyber Security"
echo ${user[role]} # Cyber Security
```

الفرق بين المصفوفة المفهرسة والمصفوفة الترابطية

المصفوفة الترابطية Associative	المصفوفة المفهرسة indexed	الميزة
عن طريق المفتاح ("name", "role", ...)	عن طريق الرقم (0, 1, 2, ...)	طريقة الوصول
declare -A arr_name	arr=(element1 element2 ...) نفصل بين العناصر بفاصل	طريقة التعريف
أزواج المفتاح والقيمة (key-value pairs) الإعدادات (configs)	القوائم (lists) القوائم المنسدلة (menus)	حالات الاستخدام
<code>\${arr["role"]}</code>	<code>\${arr[1]}</code>	مثال للوصول

العمليات الشائعة على المصفوفات

المصفوفة الترابطية Associative	المصفوفة المفهرسة indexed	العملية
<code>user[role]="Cyber Security"</code>	<code>arr+=("cherry")</code>	إضافة عنصر
<code>user[role]="DevOps"</code>	<code>arr[1]="blueberry"</code>	تحديث قيمة عنصر
<code>echo \${#user[@]}</code>	<code>echo \${#arr[@]}</code>	معرفة عدد العناصر
للمرور على ال keys: <pre>for key in "\${!user[@]}"; do echo "\$key => \${user[\$key]}"; done</pre> للمرور على ال values: <pre>for val in "\${user[@]}"; do echo "\$val"; done</pre>	<pre>for item in "\${arr[@]}"; do echo \$item; done</pre>	المرور على عناصر المصفوفة
<code>unset user[role]</code>	<code>unset arr[2]</code>	حذف عنصر

ملاحظة حول المصفوفة الترابطية (Associative Array)

إضافة عنصر وتحديث قيمته يتمان بنفس الطريقة: `user[key]=value`

السلوك حسب حالة المفتاح (key):

- إذا كان المفتاح موجوداً مسبقاً، يتم تحديث القيمة المرتبطة به.
- إذا كان المفتاح غير موجود، يتم إضافة عنصر جديد بالمفتاح والقيمة المحددة.

■ قبل التطرق للأمثلة, سنقوم أولاً بشرح التعليمة `systemctl list-unit-files`, وذلك لتمكيننا من فهم السكريبتات المستخدمة لاحقاً بصورة أوضح.

Systemctl: هي أداة أساسية في أنظمة Linux الحديثة التي تستخدم `systemd`, وتستخدم للتحكم وإدارة خدمات النظام (system services) والوحدات (units).
الخيار `list-unit-files` يُستخدم مع `systemctl` لعرض جميع ملفات الوحدات (unit files) المتاحة على النظام، وليس الخدمات فقط.

الوحدة (unit): ملف يصف خدمة أو مهمة أو جهاز يمكن لـ `systemd` التحكم فيه.
أنواع الوحدات (Units) تشمل:

- Service: الخدمات التي تعمل في الخلفية (مثل `sshd`, `nginx`).
- Socket: نقاط الاتصال التي تنتظر الاتصالات.

■ Target: مجموعة من الخدمات مرتبطة ببعضها `mount`, `automount`, `device`, `timer` وغيرها.

وعند استخدام: `systemctl list-unit-files --type=service` يعرض كل ملفات الخدمات (service files.*) مع حالتها على النظام.

UNIT FILE	STATE	PRESET
accounts-daemon.service	enabled	enabled
alsa-restore.service	static	-
alsa-state.service	static	-
alsa-utils.service	masked	enabled
anacron.service	enabled	enabled
apparmor.service	enabled	enabled
apport-autoreport.service	static	-
apport-coredump-hook@.service	static	-
apport-forward@.service	static	-
apport.service	enabled	enabled
apt-daily-upgrade.service	static	-
apt-daily.service	static	-
apt-news.service	static	-
autovt@.service	alias	-
avahi-daemon.service	enabled	enabled
bluetooth.service	enabled	enabled
bolt.service	static	-
brltty-udev.service	static	-
brltty.service	disabled	enabled
cloud-config.service	enabled	enabled
cloud-final.service	enabled	enabled
cloud-init-hotplugd.service	static	-

■ الحالات الممكنة لملفات الخدمات:

- Enabled: الخدمة مفعلة، وستعمل تلقائياً عند بدء التشغيل..
- Disabled: الخدمة غير مفعلة، ولن تعمل تلقائياً عند بدء التشغيل.
- Masked: الخدمة مٌقفلة تماماً، حتى لو حاول المستخدم تشغيلها يدوياً فلن تعمل.
- Enabled-runtime: الخدمة مفعلة مؤقتاً في جلسة التشغيل الحالية فقط، وستتوقف بعد إعادة التشغيل
- Static: الخدمة لا يمكن تمكينها مباشرة، لكنها يمكن أن تعمل عند طلب خدمات أخرى أو عند الحاجة (تستدعى تلقائياً).

لدينا خدمة monitoring وخدمة alarm، خدمة monitoring تقوم بمراقبة النظام، وعندما تكتشف خطأ (مثل خطأ أو سلوك غير طبيعي) تقوم systemd بتشغيل خدمة alarm تلقائياً لإرسال تنبيه، هنا تكون خدمة alarm غالباً static، لأنها لا تعمل تلقائياً عند بدء التشغيل، لكنها تستدعى عند الحاجة.

بالنسبة للخيارات الأخرى مع العملية:

- --no-pager : يُستخدم لعرض ناتج الأمر كاملاً في ال terminal دون تقسيمه إلى صفحات.
- --no-legend : يُستخدم لعرض الناتج بدون إظهار عناوين الأعمدة (رؤوس الجدول).

```
rbcs@ -VMware-Virtual-Platform:~$ systemctl list-unit-files --type=service --no-
pager --no-legend
accounts-daemon.service          enabled          enabled
alsa-restore.service            static          -
alsa-state.service              static          -
alsa-utils.service              masked          enabled
anacron.service                 enabled         enabled
apparmor.service                enabled         enabled
apport-autoreport.service        static          -
apport-coredump-hook@.service    static          -
apport-forward@.service          static          -
apport.service                  enabled         enabled
apt-daily-upgrade.service        static          -
apt-daily.service               static          -
apt-news.service                static          -
autovt@.service                 alias           -
avahi-daemon.service            enabled         enabled
bluetooth.service              enabled         enabled
bolt.service                    static          -
brltty-udev.service             static          -
brltty.service                  disabled        enabled
cloud-config.service            enabled         enabled
cloud-final.service             enabled         enabled
cloud-init-hotplugd.service      static          -
cloud-init-local.service         enabled         enabled
cloud-init.service              enabled         enabled
colord.service                  static          -
configure-printer@.service       static          -
console-getty.service           disabled        disabled
```

مثال تطبيقي 1:

يقوم هذا السكريبت ب جلب جميع الخدمات (services) الموجودة في النظام، ثم يقوم بتصنيفها (فلترتها) حسب حالتها، وتخزين ناتج هذه العملية في ملف نصي.

```
#!/bin/bash
result="services_report.txt" # same file every time
while true; do
  declare -A services
  # Read service name and state only
  while read -r name state preset; do
    services["$name"]="$state"
  done << (systemctl list-unit-files --type=service --no-pager --no-legend)

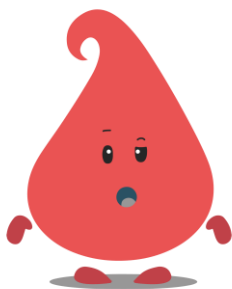
  {
    echo "===== Service Report $(date +"%Y-%m-%d %H:%M:%S") ====="
    echo "-----Enabled Services-----"
    for name in "${!services[@]}"; do
      [[ "${services[$name]}" == "enabled" ]] && echo "$name"
    done
    echo "-----Disabled Services-----"
    for name in "${!services[@]}"; do
      [[ "${services[$name]}" == "disabled" ]] && echo "$name"
    done
    echo "-----Static Services-----"
    for name in "${!services[@]}"; do
      [[ "${services[$name]}" == "static" ]] && echo "$name"
    done
    echo "-----Masked Services-----"
    for name in "${!services[@]}"; do
      [[ "${services[$name]}" == "masked" ]] && echo "$name"
    done
    echo "-----Enabled-runtime Services-----"
    for name in "${!services[@]}"; do
      [[ "${services[$name]}" == "enabled-runtime" ]] && echo "$name"
    done
  } >> "$result"
  echo "Results appended to $result at $(date +"%H:%M:%S")"
  sleep 5
done
```

1. يتم أولاً تخزين اسم ملف التقرير في المتغير "result=services_report.txt" ويستخدم هذا الملف لتخزين ناتج الفترة في كل دورة من دورات التنفيذ.
2. يستخدم السكربت حلقة while لا نهائية، حيث لا يتوقف السكربت إلا عند إيقافه يدوياً من قبل المستخدم، والهدف من ذلك هو تكرار عملية المراقبة بشكل دوري.
3. داخل الحلقة، يتم تعريف مصفوفة ترابطية باسم services هذه المصفوفة ستستخدم لتخزين: اسم الخدمة كمفتاح (Key) وحالة الخدمة كقيمة (Value).
4. يتم استخدام الأمر الذي شرحناه سابقاً لجلب خدمات النظام:
systemctl list-unit-files --type=service --no-pager --no-legend
 - ناتج هذا الأمر يتكوّن من ثلاثة أعمدة: اسم الخدمة، حالة الخدمة، preset.
 - يُعتبر ناتج هذا الأمر دخلاً (STDIN) لحلقة while.
5. تقوم حلقة while براءة الدخل سطرًا سطرًا
 - يتم تقسيم كل سطر إلى ثلاثة متغيرات: name, state, preset.
 - ويتم تخزين اسم الخدمة كمفتاح، وحالتها كقيمة داخل المصفوفة services كما ذكرنا سابقاً.
6. بعد الانتهاء من قراءة البيانات وتخزينها في المصفوفة، يتم توجيه ناتج مجموعة من الأوامر إلى الملف result باستخدام: "\$result" >> {...}، هذا يعني أن خرج جميع الأوامر داخل الأقواس سيتم إضافته إلى الملف.
7. داخل هذه الأقواس:
 - يتم طباعة تاريخ ووقت تنفيذ العملية.
 - يتم المرور على جميع مفاتيح المصفوفة services.
 - إذا كانت قيمة الخدمة enabled يتم طباعتها تحت عنوان enabled، ويتم التعامل مع باقي قيم الخدمة بنفس الطريقة.
 - الطباعة لا تظهر على الشاشة، بل تُخزّن داخل ملف التقرير.
8. بعد الانتهاء من الفترة، يتم طباعة رسالة تفيد بأن العملية تمت بنجاح، ويتوقف السكربت لمدة 5 ثوانٍ باستخدام sleep، ثم تبدأ الدورة من جديد حتى يتم إيقاف السكربت يدوياً.

■ شرح الصيغة (command) < done

- الرمز < الأول يشير إلى إعادة توجيه الإدخال القياسي (STDIN) إلى الحلقة.
- الصيغة (command) < تسمى Process Substitution:

- يتم أخذ ناتج الأمر داخل القوسين.
- تحويله إلى ملف مؤقت.
- تمرير هذا الملف إلى الحلقة وكأنه ملف نصي عادي.



لماذا لا نستخدم | pipe بدلاً من ذلك؟

لو استخدمنا:

systemctl list-unit-files ... | while read -r name state preset; do

في Bash، الحلقة ستعمل في Subshell، بالتالي أي متغير يتم تعديله داخل الحلقة لن يكون متاحاً خارجها، وبما أننا نملأ مصفوفة services ونحتاجها لاحقاً، فإن هذا لن يعمل كما هو متوقع.

أما باستخدام (command) < فإن الحلقة تعمل في نفس الـ shell الحالي، والمصفوفة services تبقى متاحة بعد انتهاء الحلقة.

تطبيق السكربت

```
root@master:/home/mohammad/scripts# ./array.sh
Results appended to services_report.txt at 19:11:39
Results appended to services_report.txt at 19:11:45
Results appended to services_report.txt at 19:11:50
^C
root@master:/home/mohammad/scripts# cat services_report.txt
===== Service Report 2025-11-28 19:11:20 =====
-----Enabled Services-----
blk-availability.service
vmtoolsd.service
systemd-timesyncd.service
lxd-agent-9p.service
cloud-init.service
ubuntu-advantage.service
iscsi.service
snapd.system-shutdown.service
vgauth.service
pollinate.service
lvm2-monitor.service
dbus-org.freedesktop.timesync1.service
syslog.service
getty@.service
open-iscsi.service
cloud-config.service
secureboot-db.service
snapd.service
networkd-dispatcher.service
ua-reboot-cmds.service
```

مفهوم File Rotation (تدوير الملفات)

فكرة File Rotation تقوم على إنشاء ملفات متعددة لنفس النوع من البيانات (مثل التقارير أو النسخ الاحتياطية)، مع الاحتفاظ بعدد محدد منها فقط.

والهدف الأساسي من هذه الآلية هو توفير مساحة التخزين، منع امتلاء القرص، تسهيل إدارة الملفات القديمة.

■ مثال توضيحي

عند تنفيذ عملية Backup بشكل متكرر، لا نحتاج غالباً إلى الاحتفاظ بجميع النسخ السابقة، حيث في كثير من الحالات، يكفي الاحتفاظ بأخر نسختين فقط، وحذف النسخ الأقدم.

آليات تطبيق File Rotation:

يمكن تنفيذ عملية التدوير بعدة طرق، من أشهرها:

- التدوير بناءً على الحجم (Size-based Rotation).
- التدوير بناءً على التاريخ أو الوقت (Time-based Rotation) وهي الأكثر استخداماً.
- التدوير بناءً على عدد الملفات (Count-based Rotation).

مثال تطبيقي 2:

في هذا المثال، يقوم السكريبت بجلب الخدمات وتصنيفها وتخزينها، لكن مع اختلاف عن المثال السابق:

- يتم إنشاء ملف جديد في كل دورة بدلاً من إضافة النتائج إلى نفس الملف.
- جميع الملفات تُخزن في مجلد مخصص.
- يتم تطبيق عملية التدوير (Rotation) بحيث يبقى المجلد يحتوي فقط على آخر ملفين.

```
#!/bin/bash
rotation_dir="/home/mohammad/scripts/rotation"
mkdir -p "$rotation_dir" # create folder if not exists
while true; do
  declare -A services
  # Read service name and state
  while read -r name state preset; do
    services["$name"]="$state"
  done < <(systemctl list-unit-files --type=service --no-pager --no-legend)
  # --- Create new rotated file with timestamp ---
  TIMESTAMP=$(date +"%Y-%m-%d_%H-%M-%S")
  NEW_FILE="$rotation_dir/services_report_${TIMESTAMP}.txt"
  {
    echo "===== Service Report $(date +"%Y-%m-%d %H:%M:%S") ====="
    echo "-----Enabled Services-----"
    for name in "${!services[@]}"; do
      [[ "${services[$name]}" == "enabled" ]] && echo "$name"
    done

    echo "-----Disabled Services-----"
    for name in "${!services[@]}"; do
      [[ "${services[$name]}" == "disabled" ]] && echo "$name"
    done

    echo "-----Static Services-----"
    for name in "${!services[@]}"; do
      [[ "${services[$name]}" == "static" ]] && echo "$name"
    done

    echo "-----Masked Services-----"
    for name in "${!services[@]}"; do
      [[ "${services[$name]}" == "masked" ]] && echo "$name"
    done

    echo "-----Enabled-runtime Services-----"
    for name in "${!services[@]}"; do
      [[ "${services[$name]}" == "enabled-runtime" ]] && echo "$name"
    done
  } > "$NEW_FILE"

  echo "New report written: $NEW_FILE"
  # --- Keep only last 2 reports ---
  FILE_LIST=$(ls -t $rotation_dir/services_report_*.txt 2>/dev/null)
  COUNT=$(echo "$FILE_LIST" | wc -l)
  if [ "$COUNT" -gt 2 ]; then
    echo "$FILE_LIST" | tail -n +3 | xargs -r rm --
    echo "Old reports removed, kept the last 2 only."
  fi
  sleep 5
done
```



1. يتم تخزين المسار الكامل للمجلد الذي سيتم تخزين الملفات فيه في متغير لتسهيل الاستخدام
`rotation_dir="/home/mohammad/scripts/rotation"`
2. الأمر `mkdir -p` ينشئ المجلد وأي مجلدات أب موجودة في المسار تلقائيًا إذا لم تكن موجودة مسبقًا. يضمن هذا أن المجلد موجود قبل تخزين أي ملفات.
3. يتم تعريف مصفوفة ترابطية لتخزين الخدمات وحالتها، تمامًا كما في المثال السابق.
4. يتم إنشاء اسم ملف جديد فريد، حيث تم استخدام البصمة الزمنية (Timestamp) لتمييز كل ملف، دمج اسم المجلد مع اسم الملف ليعطي المسار الكامل للملف.
5. يتم طباعة الخدمات المصنفة في الملف الجديد تمامًا كما في المثال السابق.
6. والآن نأتي لعملية التدوير، هدفنا منها الاحتفاظ فقط بآخر ملفين في المجلد، وحذف الملفات الأقدم.

```
FILE_LIST=$(ls -t ${rotation_dir}/services_report_*.txt 2>/dev/null)
ls -t يترتب الملفات من الأحدث إلى الأقدم، وعملية إعادة توجيه الخطأ 2>/dev/null تمنع ظهور رسائل الخطأ في حال عدم وجود ملفات.
```

```
COUNT=$(echo "$FILE_LIST" | wc -l)
```

يُنفذ هذا السطر عندما يكون عدد الملفات الموجودة أكبر تمامًا من 2.

- `tail -n +3`: يبدأ من السطر الثالث من قائمة الملفات المرتبة، أي أنه يختار الملفات الأقدم بعد أول ملفين (الأحدث)، ليتم حذفها أثناء عملية التدوير
- `xargs`: يستخدم لأخذ مخرجات الأمر السابق وتحويلها إلى مدخلات للأمر `rm`
- `-r`: يمنع التنفيذ إذا لم توجد ملفات للحذف
- `--`: يضمن أن أسماء الملفات التي تبدأ ب (-) تُعامل كملفات وليس كخيارات للأمر `rm`

تنفيذ السكريبت:

- عند تشغيل السكريبت لأول مرة:

```
root@master:/home/mohammad/scripts# ./rotation.sh
New report written: /home/mohammad/scripts/rotation/services_report_2025-11-29_07-21-55.txt
New report written: /home/mohammad/scripts/rotation/services_report_2025-11-29_07-22-01.txt
New report written: /home/mohammad/scripts/rotation/services_report_2025-11-29_07-22-06.txt
Old reports removed, kept the last 2 only.
New report written: /home/mohammad/scripts/rotation/services_report_2025-11-29_07-22-12.txt
Old reports removed, kept the last 2 only.
New report written: /home/mohammad/scripts/rotation/services_report_2025-11-29_07-22-18.txt
Old reports removed, kept the last 2 only.
^C
```

نلاحظ أن رسالة حذف الملفات القديمة ظهرت لأول مرة بعد إنشاء التقرير الثالث، لأن عدد الملفات تجاوز الحد المسموح به (2 ملف). بعد ذلك، أصبح السكريبت يقوم بالحذف بعد كل إضافة جديدة تلقائيًا، للحفاظ على أن يحتوي المجلد دائمًا على أحدث ملفين فقط.

- محتويات المجلد الذي تُخزن فيه الملفات بعد تنفيذ السكريبت:

```
root@master:/home/mohammad/scripts# ls rotation
services_report_2025-11-29_07-22-22.txt services_report_2025-11-29_07-22-28.txt
root@master:/home/mohammad/scripts#
```


هذا المثال نفس المثال السابق، لكن نقوم هنا أيضاً بإرسال رسالة عن طريق بوت تلغرام بتفاصيل الخدمات.

```
#!/bin/bash
# === Telegram Bot Settings ===
BOT_TOKEN="xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
CHAT_ID="xxxxxxxxxxxxxx"
# === Rotation Settings ===
rotation_dir="/home/mohammad/scripts/rotation"
mkdir -p "$rotation_dir" # create folder if not exists
while true; do
  declare -A services
  # Read service name and state
  while read -r name state preset; do
    services["$name"]="$state"
  done < <(systemctl list-unit-files --type=service --no-pager --no-legend)
  # --- Count each service state ---
  ENABLED=0
  DISABLED=0
  STATIC=0
  MASKED=0
  ENABLED_RUNTIME=0
  for state in "${services[@]"; do
    case "$state" in
      enabled) ((ENABLED++)) ;;
      disabled) ((DISABLED++)) ;;
      static) ((STATIC++)) ;;
      masked) ((MASKED++)) ;;
      enabled-runtime) ((ENABLED_RUNTIME++)) ;;
    esac
  done
  FAILED=$(systemctl list-units --state=failed | grep -c 'failed')
  # --- Create new rotated file with timestamp ---
  TIMESTAMP=$(date +%Y-%m-%d %H:%M:%S)
  NEW_FILE="$rotation_dir/services_report_${TIMESTAMP}.txt"
  {
    echo "===== Service Report $(date +%Y-%m-%d %H:%M:%S) ====="
    echo "-----Enabled Services-----"
    for name in "${services[@]"; do [[ "${services[$name]}" == "enabled" ]] && echo "$name"; done

    echo "-----Disabled Services-----"
    for name in "${services[@]"; do [[ "${services[$name]}" == "disabled" ]] && echo "$name"; done

    echo "-----Static Services-----"
    for name in "${services[@]"; do [[ "${services[$name]}" == "static" ]] && echo "$name"; done

    echo "-----Masked Services-----"
    for name in "${services[@]"; do [[ "${services[$name]}" == "masked" ]] && echo "$name"; done

    echo "-----Enabled-runtime Services-----"
    for name in "${services[@]"; do [[ "${services[$name]}" == "enabled-runtime" ]] && echo "$name"; done
  } > "$NEW_FILE"

  echo "New report written: $NEW_FILE"
  # --- Keep only last 2 reports ---
  FILE_LIST=$(ls -t $(rotation_dir)/services_report_*.txt 2>/dev/null)
  COUNT=$(echo "$FILE_LIST" | wc -l)
  if [ "$COUNT" -gt 2 ]; then
    echo "$FILE_LIST" | tail -n +3 | xargs -r rm --
    echo "Old reports removed, kept the last 2 only."
  fi
  # --- Telegram Alert with HTML formatting ---
  MESSAGE="<b> Service Report $(date +%Y-%m-%d %H:%M:%S)</b>"
  <b> Enabled:</b> ${ENABLED}
  <b> Disabled:</b> ${DISABLED}
  <b> Static:</b> ${STATIC}
  <b> Masked:</b> ${MASKED}
  <b> Enabled-runtime:</b> ${ENABLED_RUNTIME}
  <b> Failed:</b> ${FAILED}"
  curl -s -o /dev/null -X POST "https://api.telegram.org/bot${BOT_TOKEN}/sendMessage" \
    -d chat_id="${CHAT_ID}" \
    -d text="${MESSAGE}" \
    -d parse_mode="HTML"
  sleep 5
done
```

1. بداية نوضح إعدادات بوت Telegram:

- كل بوت Telegram يمتلك Token خاص به.
- كل مستخدم يتفاعل مع البوت يحصل على chat_id، وهو المعرف الذي يستخدمه البوت لتمييز المستخدمين.

لذلك نقوم بتخزين:

- CHAT_ID**: معرف المستخدم الذي نريد إرسال الرسالة إليه.



ملاحظة مهمة: يجب أن يكون المستخدم قد تفاعل مع البوت مرة واحدة على الأقل (أرسل له رسالة)، حتى يتمكن Telegram من توليد chat_id له.

2. بعد ذلك نقوم بتحديد مسار المجلد الذي ستخزن فيه ملفات التقارير (Rotation Directory), ثم نقوم بجلب جميع خدمات النظام وحالاتها, ونقوم بتخزين اسم الخدمة كمفتاح (Key) وحالتها كقيمة (Value) داخل مصفوفة ترابطية, كما في المثال السابق.

3. نقوم بتعريف متغيرات لعدد كل نوع من حالات الخدمات: enabled, disabled, static, masked, enabled-runtime, ثم نمر على قيم المصفوفة باستخدام: "for state in "\${services[@]}" عند استخدام "\${services[@]}" يتم المرور على القيم. بينما "\${!services[@]}" يتم المرور على المفاتيح.



داخل الحلقة نستخدم تعليمة case, بحيث:
كل حالة تطابق قيمة معينة.
يتم زيادة العداد الموافق لها.

4. نقوم بتخزين عدد الخدمات التي حالتها failed في متغير FAILED باستخدام الأمر:
FAILED=\$(systemctl list-units --state=failed | grep -c 'failed')
■ systemctl list-units --state=failed يعرض جميع الوحدات (Units) التي حالتها failed حالياً.
■ grep -c 'failed': يبحث عن الأسطر التي تحتوي على كلمة failed ويقوم بعدها فقط دون طباعتها (c- تعني count).

ملاحظة: يمكن استخدام wc -l بدل grep -c, لكن بشرط استخدام الخيار --no-legend مع systemctl لأن wc -l سيعد سطر العناوين وسطر الملخص (footer) أيضاً:

```
rbc@ -VMware-Virtual-Platform:~$ systemctl list-units --state=failed
UNIT LOAD ACTIVE SUB DESCRIPTION
0 loaded units listed.
rbc@ -VMware-Virtual-Platform:~$ systemctl list-units --state=failed | grep -c 'failed'
0
rbc@ -VMware-Virtual-Platform:~$ systemctl list-units --state=failed | wc -l
3
rbc@ -VMware-Virtual-Platform:~$ systemctl list-units --state=failed --no-legend | wc -l
0
```

5. نقوم بإنشاء اسم ملف فريد في كل مرة باستخدام البصمة الزمنية (Timestamp) نطبع تقرير الخدمات بعد فلترتها داخل هذا الملف, ثم نطبق عملية التدوير بحيث نحفظ بأحدث ملفين فقط, ونحذف الملفات الأقدم (آلية التدوير هنا مطابقة تماماً للمثال السابق).

6. نقوم بتخزين محتوى الرسالة في متغير MESSAGE، وذلك لتحسين قابلية قراءة السكربت، وتسهيل تعديل محتوى الرسالة لاحقاً.
تم اختيار تنسيق HTML لأنه بسيط و يسمح باستخدام التنسيقات مثل ، يمكن أيضاً استخدام تنسيقات أخرى مثل JSON أو Markdown.

7. نقوم بإرسال الرسالة فعلياً عبر الأمر: `curl -s -o /dev/null -X POST ...`

- Curl: أداة سطر أوامر لإرسال واستقبال البيانات عبر الشبكة.
- -s (silent): يمنع عرض معلومات التقدم أو الأخطاء.
- -o /dev/null: يمنع طباعة استجابة Telegram على ال terminal.
- -X POST: يحدد نوع الطلب على أنه POST (إرسال البيانات).
- parse_mode=HTML: يخبر Telegram أن نص الرسالة مكتوب بتنسيق HTML، وبدونه ستعرض الرسالة كنص عادي دون تنسيق.

طريقة إنشاء بوت التلغرام:

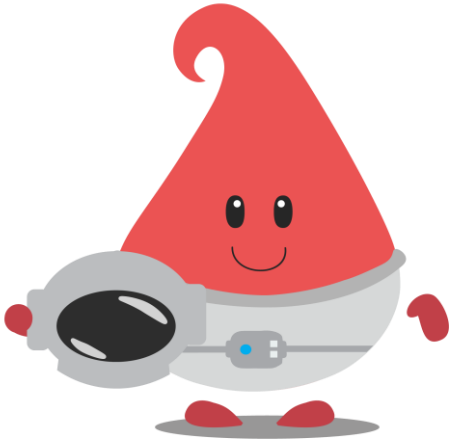
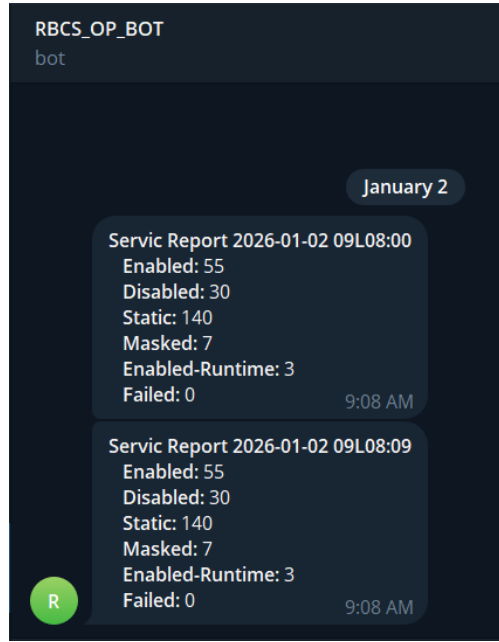
- نقوم بإنشاء البوت عبر BotFather @ واتباع التعليمات للحصول على BOT_TOKEN
 - نقوم بزيارة الموقع: (نستخدم ال Token الخاصة للبوت هنا)
- <https://api.telegram.org/bot<YOUR BOT TOKEN>/getUpdates>

- ثم نرسل رسالة للبوت الذي تم إنشاؤه لتفاعل مع البوت و نحصل على chat_id.
- نستخرج ال chat id من الموقع (قد يحتاج ل refresh).



```
{
  "update_id": 397133790,
  "message": {
    "message_id": 3,
    "from": {
      "id": 714527346,
      "is_bot": false,
      "first_name": "RBCS",
      "username": "RBCs_OP",
      "language_code": "en"
    },
    "chat": {
      "id": 714527346,
      "first_name": "RBCS",
      "username": "RBCs_OP",
      "type": "private"
    },
    "date": 1767332525,
    "text": "hh"
  }
}
```

- بعد ذلك نستخدم BOT_TOKEN و CHAT_ID داخل السكربت.



- إرسال الرسالة لأكثر من مستخدم:

هناك طريقتان:

1. استخدام مصفوفة chat_id:

- تخزين جميع معرفات المستخدمين في مصفوفة.

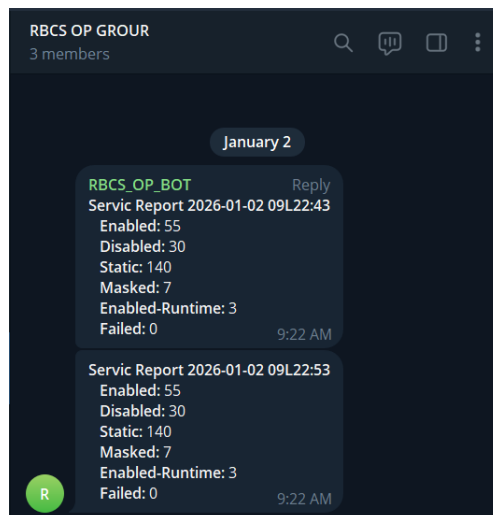
- المرور عليهم واحدًا واحدًا وإرسال الرسالة لكل منهم.

2. استخدام مجموعة (Group):

- إنشاء مجموعة Telegram.

- إضافة البوت إليها ومنحه صلاحيات الإرسال.

- إرسال الرسالة إلى chat_id الخاص بالمجموعة (جميع أعضاء المجموعة سيستقبلون الرسالة).



انتهت المحاضرة